

Clase 327 — Ingeniería de detección avanzada y validación

Parte: **17 — Profundización para certificaciones** · Fuente: *MITRE ATT&CK · SANS SEC555 / BTL1 SOC · SigmaHQ*  Duración estimada: **150 min** · Nivel: **Experto**

Objetivo

Convertir la detección de amenazas en una **disciplina de ingeniería**: tratar las reglas como código (*detection-as-code*), gestionar su ciclo de vida completo, **validarlas** con emulación de adversario (Atomic Red Team), reducir sistemáticamente los falsos positivos y medir su eficacia con métricas objetivas. Es el núcleo del trabajo de un ingeniero de detección moderno y del módulo de **operaciones de seguridad de CySA+ y SIEM de BTL1**.

Resultados de aprendizaje

Al finalizar, el alumno podrá:

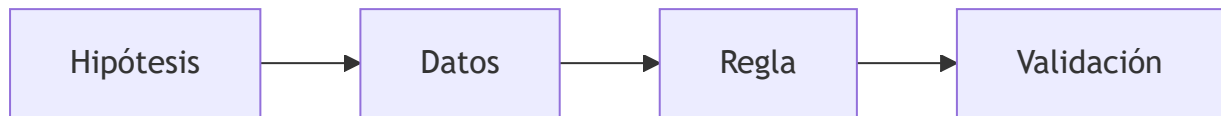
1. **Aplicar** el ciclo de vida de una detección: hipótesis → desarrollo → prueba → despliegue → mantenimiento → retiro.
2. **Escribir** reglas portables en **Sigma** y traducirlas a la sintaxis del SIEM (EQL, KQL, SPL).
3. **Gestionar** las detecciones como código: versionado en Git, revisión por pares y CI.
4. **Validar** una detección ejecutando la técnica correspondiente con **Atomic Red Team**.
5. **Reducir falsos positivos** mediante afinado, *allowlisting* y contexto, midiendo el impacto.
6. **Calcular** métricas de detección (cobertura ATT&CK, tasa de FP, MTTD) para dirigir la mejora.

Temas

#	Tema	Por qué importa
1	Detection-as-code	Reglas versionadas, revisadas y testeadas como cualquier software
2	Ciclo de vida de una detección	Evita el "cementerio de reglas" que nadie mantiene
3	Sigma como formato portable	Escribes una vez y despliegas en cualquier SIEM
4	Traducción a EQL/KQL/SPL	La regla debe correr donde vive la telemetría
5	Validación con Atomic Red Team	Sin ejecutar la técnica no sabes si la regla dispara
6	Reducción de falsos positivos	Un SOC ahogado en ruido es un SOC ciego
7	Métricas de detección	Lo que no se mide no se mejora; guía la priorización
8	Mapeo de cobertura ATT&CK	Revela puntos ciegos frente al adversario

Modelo mental y caso de decisión

Una detección vive: hipótesis, datos, lógica, pruebas, despliegue, métricas y mantenimiento frente a deriva.



Caso razonado. La regla pasa un ejemplo pero no variantes. Se añade corpus positivo y negativo.

Glosario operativo complementario

Término	Definición
Evidencia	Información cuya procedencia y relación con un criterio pueden revisarse.
Supuesto	Condición declarada de la que depende una conclusión.
Riesgo residual	Exposición que permanece después del tratamiento.

Criterio de dominio

El alumno demuestra dominio cuando explica el diagrama, resuelve el caso con evidencia, declara límites y puede defender por qué su decisión cambia si cambia un supuesto.

Definiciones y características

- **Detection-as-code:** práctica de gestionar reglas de detección con las herramientas del desarrollo de software (Git, revisión, CI/CD, pruebas). Característica clave: trazabilidad y reproducibilidad de cada cambio.
- **Sigma:** formato abierto en YAML para escribir reglas de detección independientes del SIEM. Característica clave: se traduce a EQL, SPL, KQL, etc. con `sigma-cli` /pySigma.
- **EQL (Event Query Language):** lenguaje de Elastic para expresar detecciones basadas en secuencias de eventos. Característica clave: potente para correlacionar eventos ordenados en el tiempo.
- **Atomic Red Team:** biblioteca de pruebas atómicas que emulan técnicas ATT&CK de forma pequeña y controlada. Característica clave: valida cada detección con la técnica real que debe atrapar.
- **Falso positivo (FP):** alerta que no corresponde a actividad maliciosa. Característica clave: erosiona la confianza y satura al analista; se combate con contexto y afinado.
- **Ciclo de vida de la detección:** flujo desde la idea hasta el retiro de una regla. Característica clave: incluye mantenimiento; una regla sin dueño se degrada.
- **MTTD (Mean Time To Detect):** tiempo medio en detectar un incidente. Característica clave: métrica de resultado que la ingeniería de detección busca reducir.
- **Cobertura ATT&CK:** proporción de técnicas relevantes con detección validada. Característica clave: se visualiza con ATT&CK Navigator para exponer puntos ciegos.

Herramientas y preparación

⚠ **Laboratorio aislado obligatorio.** Atomic Red Team **ejecuta técnicas ofensivas reales** (creación de tareas, volcado de credenciales simulado, etc.). Córralo únicamente en una VM de laboratorio con snapshot, aislada de la red de producción y con autorización explícita. Nunca ejecute pruebas atómicas en equipos productivos.

- **SIEM/EDR de laboratorio:** Elastic Stack (Elasticsearch + Kibana) o Splunk gratuito, con Sysmon + una configuración robusta (p. ej. la de Olaf Hartong) en el endpoint Windows.
- **Sigma + pySigma / `sigma-cli`** : para escribir y traducir reglas.
- **Atomic Red Team** (`Invoke-AtomicRedTeam` en PowerShell) para emular técnicas.
- **Git** para versionar las reglas (detection-as-code) y un pipeline de CI que valide sintaxis Sigma.
- **MITRE ATT&CK Navigator** para mapear cobertura.

Instalación mínima de Atomic Red Team en la VM de laboratorio:

```
IEX (IWR 'https://raw.githubusercontent.com/redcanaryco/invoke-atomicredteam/master/install-a  
Install-AtomicRedTeam -getAtomics
```

Laboratorio guiado

Desarrollaremos y validaremos una detección para **T1053.005 (tarea programada como persistencia)** de punta a punta.

1. **Formula la hipótesis de detección.** "Un atacante creará una tarea programada con `schtasks /create` para persistir; deberíamos verlo en Sysmon Event ID 1 (creación de proceso) o Event ID 4698 del log de seguridad."
2. **Escribe la regla en Sigma** (`t1053_005_schtasks.yml`):

```
yaml title: Persistencia via schtasks.exe status: experimental logsource: category:  
process_creation product: windows detection: selection: Image|endswith: '\schtasks.exe'  
CommandLine|contains|all: - '/create' - '/sc' condition: selection falsepositives: - Software  
de despliegue y administración legítimos level: medium tags: - attack.persistence -  
attack.t1053.005
```

3. **Versiona la regla** (detection-as-code):

```
bash git add detections/t1053_005_schtasks.yml git commit -m "feat(detección): T1053.005  
schtasks persistencia"
```

Añade en CI una validación de sintaxis con `sigma check` para que ninguna regla rota entre a la rama principal. 4. **Traduce la regla al SIEM** de destino:

```
bash sigma convert -t esql detections/t1053_005_schtasks.yml # Elastic # o sigma convert -t  
splunk detections/t1053_005_schtasks.yml # Splunk SPL
```

5. **Despliega** la consulta traducida como regla de alerta en Kibana/Splunk y confirma que está activa.
6. **Valida con Atomic Red Team** ejecutando la técnica en la VM aislada:

```
powershell Invoke-AtomicTest T1053.005 -TestNumbers 1
```

7. **Verifica que la regla dispara.** Comprueba en el SIEM que se generó la alerta con la marca de tiempo de la prueba atómica. Si **no** dispara, revisa la telemetría (¿Sysmon captura el evento?) y el mapeo de campos.
8. **Limpia la prueba** para no dejar la persistencia creada:

```
powershell Invoke-AtomicTest T1053.005 -TestNumbers 1 -Cleanup
```

9. **Reduce falsos positivos.** Revisa alertas de una semana simulada: si tu herramienta de despliegue (p. ej. SCCM) crea tareas legítimas, añade una exclusión precisa:

```
yaml filter: ParentImage|endswith: '\ccmexec.exe' condition: selection and not filter
```

Mide la tasa de FP antes y después del afinado. 10. **Registra métricas y cobertura.** Anota MTTD de la prueba, tasa de FP y marca la técnica como "cubierta y validada" en ATT&CK Navigator.

Ejercicios

1. Escribe una regla Sigma para T1059.001 (PowerShell con `-enc`) y tradúcela a EQL y SPL.
2. Configura un pipeline de CI que ejecute `sigma check` sobre el directorio de detecciones en cada commit.
3. Ejecuta tres pruebas de Atomic Red Team distintas y anota cuáles disparan tus reglas y cuáles no.
4. Toma una regla ruidosa y redúcele la tasa de FP con al menos una exclusión justificada, midiendo el antes/después.
5. Construye una capa en ATT&CK Navigator que muestre las técnicas cubiertas por tus reglas validadas.
6. Define y calcula tres métricas de detección para tu conjunto de reglas (cobertura, FP rate, MTTD).

Reto verificable

Se te pide entregar la detección de **T1547.001 (persistencia por clave Run del registro)** lista para producción. **Criterio de aceptación:** entregas (a) una regla Sigma versionada en Git que pasa `sigma check`, (b) evidencia de que dispara al ejecutar la prueba atómica correspondiente de Atomic Red Team (captura de la alerta con su timestamp), (c) al menos una exclusión de FP justificada con datos, y (d) la técnica marcada como validada en ATT&CK Navigator. El reto se logra si otro ingeniero puede clonar el repo, desplegar la regla y reproducir la validación.

⚠ Errores comunes

Síntoma / mensaje	Causa y cómo arreglar
La regla no dispara pese a ejecutar la técnica	La telemetría no captura el evento; verifica que Sysmon/EDR registra el campo usado en la regla
<code>sigma convert</code> falla o produce campos vacíos	Falta el <i>pipeline</i> de mapeo de campos del backend; especifica el pipeline correcto (<code>-p sysmon</code> , <code>-p ecs_windows</code>)
Avalancha de falsos positivos tras desplegar	Regla demasiado amplia; añade condiciones de contexto (proceso padre, usuario, ruta) en vez de bajar el nivel
La regla se "rompe" tras un cambio y nadie lo nota	Falta CI; integra <code>sigma check</code> y pruebas de regresión en el pipeline
Cobertura ATT&CK aparentemente alta pero sin detecciones reales	Se contó "tener reglas" en vez de "reglas validadas"; solo marca cobertura tras validar con Atomic Red Team
Atomic test deja artefactos de persistencia en la VM	Olvidaste <code>-Cleanup</code> ; ejecútalo siempre y restaura el snapshot

? Preguntas frecuentes

? **¿Por qué escribir en Sigma y no directo en el lenguaje del SIEM?** Sigma es portable: escribes la lógica una vez y la traduces a EQL, SPL o KQL. Evita quedar atado a un proveedor y facilita compartir reglas con la comunidad (SigmaHQ).

? **¿Detection-as-code no es sobreingeniería para un SOC pequeño?** No. Aun con pocas reglas, versionarlas en Git y validar su sintaxis en CI evita cambios silenciosos y facilita revertir una regla que rompió el flujo de alertas.

? **¿Cómo sé que una detección realmente funciona?** Ejecutando la técnica que debe atrapar. Atomic Red Team proporciona pruebas atómicas por técnica ATT&CK; si tras ejecutarla no hay alerta, la detección no sirve todavía.

? **¿Bajar el nivel de severidad reduce los falsos positivos?** No: solo esconde el problema. Los FP se reducen añadiendo contexto (proceso padre, usuario, ruta) y exclusiones precisas y medidas, no cambiando la severidad.

? **¿Qué métrica importa más?** Depende del objetivo, pero MTTD (rapidez de detección), tasa de FP (calidad) y cobertura ATT&CK validada (alcance) juntas dan una imagen equilibrada. Una sola aislada engaña.

🔗 Referencias

- MITRE ATT&CK y ATT&CK Navigator: <https://attack.mitre.org/> · <https://mitre-attack.github.io/attack-navigator/>
- Proyecto Sigma (SigmaHQ): <https://github.com/SigmaHQ/sigma>
- Atomic Red Team (Red Canary): <https://github.com/redcanaryco/atomic-red-team>
- Palantir — Alerting and Detection Strategy Framework: <https://github.com/palantir/alerting-detection-strategy-framework>

- SANS SEC555 *SIEM with Tactical Analytics*; certificación CompTIA CySA+ (dominio Security Operations).



Material descargable

-  [Guía en PDF](#) — versión imprimible de esta clase.
-  [Presentación \(PPTX\)](#) — deck para proyectar en clase.



Clase anterior

[Clase 326 — Análisis de malware para respuesta a incidentes](#)



Siguiente clase

[Clase 328 — Gestión de riesgos cuantitativa y continuidad avanzada](#)